

미분류 파일 자동 정리 시스템

설계 문서 — Claude Code 핸드오프용

작성일: 2026-07-09

1. 배경 및 목표

기존에 폴더 구조로 정리해 둔 대량의 파일이 있고, 이후 분류되지 않은 채 하나의 폴더에 쌓인 파일들을 기존 분류 체계에 맞춰 자동으로 이동시키는 프로그램이 필요합니다. 다만 Claude Code가 학습한 분류 체계가 실제로 올바른지, 그리고 이동이 실제로 적절한지 사람이 사전에 확인할 수 있어야 하며, 파일 이동은 되돌리기 어려운 작업이므로 실행 전 검증 단계와 실행 후 롤백 수단이 반드시 필요합니다.

이 문서는 다음 세 가지 문제를 해결하기 위한 설계를 담고 있습니다:

- 판단(분류 로직)과 실행(파일 이동)을 분리하여, 실행 전 검토 지점을 확보
- 폴더당 파일 수백 개 규모에서도 대표성 있게 명명 패턴을 학습하는 샘플링 방법
- 파일 수가 많아 전수 검토가 불가능한 상황에서, 사람이 실질적으로 확인 가능한 검증 방법

2. 전체 파이프라인 개요

전체 과정은 4단계로 구성되며, 각 단계의 산출물은 다음 단계의 입력이 되는 파일(JSON/Markdown/Excel)입니다. 실제 파일 이동은 마지막 4단계에서만 발생합니다.

단계	스크립트	입력	산출물	사람의 역할
1. 학습	learn_taxonomy.py	기존 분류 폴더 전체	classification_rules.md	규칙이 실제 분류 기준과 맞는지 1차 확인
2. 계획	make_plan.py	classification_rules.md + 미분류 폴더	move_plan.xlsx	(3단계에서 검증)
3. 검증	validate_plan.py	move_plan.xlsx + 기존 분류	검증 리포트 (정확도 통계)	애매한 항목만

단계	스크립트	입력	산출물	사람의 역할
		폴더(홀드아웃)		집중 검토·승인
4. 실행	execute_plan.py	승인된 move_plan.xlsx	실제 파일 이동 + move_log.json	이상 시 undo.py로 즉시 복구

3. 1 단계 — 분류 체계 학습 (learn_taxonomy.py)

폴더당 파일이 수백 개일 수 있으므로, 앞쪽 10~20개만 고정 샘플링하면 편향된 학습이 됩니다. 대신 폴더 내부에서 명명 패턴을 먼저 자동으로 군집화한 뒤, 군집 수에 비례해서 샘플을 추출합니다.

3.1 처리 순서

1. 각 폴더 내 파일명을 정규식 기반 특징(확장자, 날짜 포맷 유무, 구분자 종류, 자릿수 패턴 등)으로 군집화
2. 각 군집에서 대표 파일명 3~5 개씩만 추출 (군집이 많으면 샘플 수도 자동으로 늘어남)
3. 어떤 군집에도 속하지 않는 이상치(outlier) 파일명은 샘플링하지 않고 전량 포함
4. 파일명 텍스트는 샘플링하되, 총 파일 수·확장자별 개수·군집별 개수 같은 통계는 항상 전체 스캔 결과를 그대로 반영
5. 이 결과를 JSON 으로 저장한 뒤 Claude Code 가 읽어 classification_rules.md 작성

3.2 classification_rules.md 예시 형식

각 폴더 규칙에는 반드시 커버리지(적용 범위) 정보를 함께 기재하여, 규칙이 실제 폴더를 얼마나 설명하는지 검토 시점에 바로 확인할 수 있게 합니다.

```
## 재무/리포트 폴더 (총 312개)
- 패턴 A "종목명_날짜.pdf": 268개
- 패턴 B "종목명(날짜)_애널리스트.xlsx": 31개
- 미매칭: 13개 → 목록은 문서 하단 부록 참조
```

→ 이 문서만 읽고 최상길님이 분류 매커니즘을 제대로 반영했는지 파일을 하나도 옮기기 전에 확인하실 수 있습니다. 규칙이 틀렸다면 이 문서를 직접 수정하고, 이후 단계는 이 문서를 기준으로만 동작합니다.

4. 2 단계 — 이동 계획 생성 (make_plan.py)

classification_rules.md를 기준으로 미분류 폴더의 파일을 분류하되, 이 단계에서는 실제 이동을 하지 않고 move_plan.xlsx만 생성합니다.

파일명	제안 폴더	확신도	분류 근거	승인(Y/N)
xxx.pdf	재무/리포트	높음	파일명에 종목명+날짜 패턴 일치	Y
yyy.hwp	???	낮음	매칭 규칙 없음	N

확신도는 매칭된 패턴이 해당 폴더의 커버리지 상위 패턴인지, 아니면 이상치에 가까운지를 기준으로 산정합니다.

5. 3 단계 — 검증: 전수 확인 없이 신뢰도 확보하기

파일이 많으면 move_plan.xlsx를 처음부터 끝까지 읽는 것 자체가 비현실적입니다. 따라서 검증의 핵심은 사람이 파일을 읽는 대신, 분류 로직의 정확도를 자동으로 측정 가능한 숫자로 바꾸는 것입니다.

5.1 홀드아웃 검증 (가장 중요)

이미 정리되어 있는 기존 폴더 자체가 사실상 정답 데이터입니다. 이를 활용해 사람 없이 자동 채점이 가능합니다.

- 이미 분류된 폴더에서 파일의 일부(예: 20%)를 무작위로 골라 미분류 상태인 것처럼 위장
- 학습된 규칙으로 그 파일들을 다시 분류 실행
- 원래 있던 폴더와 일치하는지 스크립트가 자동 비교 → 정확도(%) 산출
- 폴더별로 쪼개서 표시: 예) "재무리포트 폴더 정확도 96%, 회의록 폴더 62%" → 약한 규칙을 바로 특정 가능

5.2 확신도 기반 트리아지

- 홀드아웃 정확도가 높게 나온 규칙 → 해당 규칙으로 분류된 파일은 자동 승인 대상
- 확신도 낮음/애매한 파일만 사람이 검토 → 전체의 10~20% 수준으로 검토량이 줄어듦

5.3 집계 단위 검토

- 파일 단위가 아니라 "OOO 폴더에 47 개 유입 예정" 같은 폴더별 요약표만 확인
- 이동 전/후 폴더별 파일 수 분포를 비교해 비정상적으로 몰리거나 튀는 폴더만 드릴다운

5.4 무작위 스팟체크

전수 검사 대신 매 배치마다 5~10개만 무작위 추출하여 실제로 맞는지 확인합니다. 통계적 신뢰구간 개념으로, 전체가 아니라 표본만으로도 이상 여부를 충분히 감지할 수 있습니다.

5.5 점진적 적용

확신도가 가장 높은 파일부터 소량 배치(예: 50개)로 먼저 실행하고 결과를 확인한 뒤, 문제가 없으면 다음 배치로 범위를 넓혀 갑니다.

이 다섯 가지를 조합하면, 실제로 사람이 눈으로 확인해야 하는 파일 수는 전체 파일 수와 무관하게 항상 수십~백여 건 수준으로 고정됩니다.

6. 4 단계 — 실행 및 롤백 (execute_plan.py / undo.py)

- move_plan.xlsx 에서 승인(Y) 표시된 행만 실제로 이동 (shutil.move)
 - 모든 이동 내역을 move_log.json 에 기록: 원본 경로 → 새 경로, 타임스탬프
 - undo.py 실행 시 로그를 역순으로 읽어 전부 원래 위치로 복구 가능
- 이 로그가 있는 한, 최종 적용 이후에 문제를 발견하더라도 언제든지 되돌릴 수 있습니다.

7. Claude Code 역할 분담 원칙

KOSPI 모니터링 시스템에서 사용하신 하이브리드 패턴과 동일한 원칙을 적용합니다: LLM은 판단이 필요한 부분에만 쓰고, 실제 파일 조작은 결정론적 Python 스크립트로 고정합니다.

구분	담당	이유
명명 패턴 군집화, 통계 집계	Python (정규식/스크립트)	재현 가능하고 빠르며 비용이 들지 않음
규칙 문서(rules.md) 작성	Claude Code	패턴을 사람이 읽을 규칙으로 요약하는 데 강점
애매한 파일의 폴더 판단	Claude Code	이름만으로 애매한 경우 맥락적 추론 필요
실제 파일 이동, 롤백	Python (shutil)	결정론적 실행이 보장되어야 하는 위험 작업

CLAUDE.md에는 다음과 같은 안전 규칙을 명시해 두는 것을 권장드립니다:

- 어떤 경우에도 `shutil.move`를 직접 호출하지 말 것
- 분류 결과는 반드시 `move_plan.xlsx` 산출까지만 수행
- 실제 이동은 사용자가 승인한 `execute_plan.py` 실행으로만 발생

8. 폴더/파일 구조 제안

```
sort_project/
├── learn_taxonomy.py
├── make_plan.py
├── validate_plan.py
├── execute_plan.py
├── undo.py
├── CLAUDE.md
├── output/
│   ├── taxonomy_scan.json      (1단계 원본 스캔 결과)
│   ├── classification_rules.md (1단계 산출물, 검토 대상)
│   ├── move_plan.xlsx          (2단계 산출물, 승인 대상)
│   ├── validation_report.md    (3단계 홀드아웃 검증 결과)
│   └── move_log.json           (4단계 실행 로그, 롤백용)
```

9. 다음 단계 제안

10. 먼저 실제 폴더 구조를 스캔하는 learn_taxonomy.py 부터 Claude Code 에서 구현
11. classification_rules.md 를 함께 검토하며 규칙 품질을 1 차 확정
12. 홀드아웃 검증 스크립트로 정확도를 측정해 규칙을 튜닝
13. 소규모 파일럿(50 개 내외)으로 전체 파이프라인을 1 회 리허설 후 본 실행 범위 확대